

Chatnalyxer: Asynchronous Task Extraction and Priority Triage in Multi-Modal Chat Networks

Pamula Rahul, Duggina Yaswanth Chowdary, Kyatham Aishwarya, Mahira Syed
Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning)
Parul University
Vadodara, Gujarat, India

Abstract—Instant messaging (IM) platforms have become primary channels for distributing time-sensitive information in both academic and organizational contexts. Despite this, productivity tooling has not kept pace: users are left to manually identify, extract, and transfer actionable tasks from raw chat streams into structured scheduling applications, a process that is both error-prone and cognitively taxing. We present Chatnalyxer, a fully passive, multi-modal pipeline that processes WhatsApp messages at the network edge, extracts temporal tasks using a Large Language Model (LLM), and schedules them automatically without any user interaction. A key engineering decision—decoupling the Node.js edge ingestion layer from the FastAPI processing core through an asynchronous HTTP 202 WebHook—bounds local ingestion latency to $\mathcal{O}(1)$ under burst traffic, mitigating the TCP socket exhaustion that plagues synchronous architectures. To handle the deictic temporal language prevalent in chat (e.g., “tomorrow at 5”), we embed a real-time UTC timestamp into each LLM prompt, enabling precise ISO-8601 deadline resolution without any model fine-tuning. Non-text media—voice notes and images—are normalized into UTF-8 through Azure Cognitive Services (ASR and OCR, respectively) before reaching the inference layer. Evaluated against the Chatnalyxer Evaluation Corpus (CEC-10K), the pipeline achieves an F1 score of 0.94 on plain-text extraction, demonstrating that ambient, in-stream productivity management is architecturally feasible.

Index Terms—Natural Language Processing, Large Language Models, Multi-modal Extraction, WhatsApp, Edge Computing.

I. INTRODUCTION

Over the past decade, instant messaging (IM) platforms have quietly displaced email as the dominant medium for distributing organizational and academic logistics. WhatsApp alone processes over 100 billion messages daily [1], a considerable fraction of which contain deadlines, meeting requests, and shared responsibilities. The irony is structural: IM interfaces are designed around chronological recency, not semantic importance. A critical deadline buried three screens deep in a group chat is functionally invisible within hours.

Research Prototype Disclaimer: Chatnalyxer is presented as an academic research prototype intended to investigate automated task extraction from instant messaging platforms. It is not proposed as a production-ready deployment, and additional engineering, security, and compliance work would be required before real-world adoption.

The standard remediation—manually reading each message and copying tasks into Google Calendar or Microsoft To Do—is both fragile and disruptive. It requires sustained attention to multiple channels simultaneously, and any momentary

lapse in monitoring results in missed obligations. What is needed is a system that operates continuously in the background, treating task extraction as an infrastructure concern rather than a user responsibility.

We address this with **Chatnalyxer**, a passive pipeline that monitors WhatsApp at the network edge and automatically converts conversational messages into structured, scheduled tasks. Three specific technical problems motivate the design:

- **Decoupled Ingestion Architecture:** Processing LLM inference synchronously on each inbound message is architecturally untenable under bursty traffic. We decouple the edge ingestion layer from the NLP core via an HTTP 202 WebHook, so the edge immediately acknowledges receipt and the heavyweight inference executes asynchronously.
- **UTC-Anchored Temporal Resolution:** LLMs have no grounding in calendar time and cannot reliably resolve deictic expressions (e.g., “next Tuesday”) without an explicit temporal reference. We inject the message’s transmission timestamp directly into the prompt, converting relative expressions into absolute ISO-8601 deadlines without fine-tuning.
- **Multi-Modal Priority Triage:** Tasks arrive not only as text but embedded in images and voice notes. Treating every extracted item with equal urgency leads to alert fatigue. We introduce a heuristic scoring function that assigns priority based on deadline proximity, explicit urgency markers, and input modality.

Section II surveys prior work in information extraction, temporal reasoning, and academic baseline architectures. Section III describes the overall system design. Algorithmic and mathematical details follow in Sections VII through X. Experimental methodology, results, and ablation findings are reported in Sections XII through XIV. We close with a discussion of validity threats, future directions, and conclusions.

II. RELATED WORK

The technical foundations of Chatnalyxer span five established research domains: information extraction from informal text, temporal reasoning in conversational settings, multi-modal AI (ASR and OCR), retrieval-augmented generation, and privacy-preserving architectures. This section surveys the relevant literature within each domain, identifies the limitations that persist in current approaches, and motivates the specific design decisions made in Chatnalyxer.

A. Information Extraction and Named Entity Recognition in Informal Text

Information Extraction (IE) has a decades-long history, but the application of IE techniques to informal, conversational text remains an open challenge. Early systems relied on handcrafted grammars and gazetteers [2], which achieve high precision on narrow, well-formatted domains but generalize poorly to colloquial language. The transition to statistical sequence labeling—particularly Bi-LSTM-CRF architectures [3]—substantially improved recall on formal corpora; however, instant messaging text violates the lexical and structural assumptions of these models almost universally: utterances are short, punctuation is absent or non-standard, and entities are frequently implied by context rather than named explicitly.

Transformer-based encoders, beginning with BERT [4] and its successors such as RoBERTa [5], partially mitigate these limitations by capturing long-range bidirectional context. Studies on social media NER benchmarks such as WNUT-17 [6] have shown that transformer models substantially outperform BiLSTM-CRF approaches on noisy user-generated text. Nevertheless, even highly competitive NER pipelines face a fundamental limitation in this setting: they operate on fixed entity type taxonomies and require domain-specific training data to generalize. Constructing such datasets for the open-ended diversity of group chat conversations—where tasks may be phrased as questions, imperatives, or offhand references—is prohibitively expensive.

Recent work has framed extraction as a generative task to avoid this dependency. DEGREE [7] formulates event extraction as a conditional text generation problem using template-based prompting, enabling extraction without entity-type-specific training data. Similarly, large language models operating under zero-shot and few-shot prompting regimes (surveyed in [8]) have been shown to extract structured information from free text with competitive accuracy compared to fine-tuned NER models, provided the output format is tightly constrained through the prompt. Chatnalyxer adopts this generative paradigm, constraining the LLM’s output via a typed JSON schema rather than relying on a trained extraction model.

B. Temporal Expression Recognition and Normalization

Temporal information extraction has been formalized through the TimeML annotation framework [9] and evaluated across successive TempEval shared tasks [10]. Tools such as SUTime [11] and HeidelTime [12] achieve strong results on formal, document-level text by anchoring temporal expressions to a known reference date derived from the document’s meta-data. However, both systems are rule-based and depend on explicit temporal markers (e.g., “March 15, 2024”) for reliable resolution.

The dominant form of temporal language in instant messaging is deictic rather than absolute: expressions like “tomorrow”, “by EOD”, “after class”, or “this weekend” carry no calendar-specific meaning independent of the speaker’s position in time at the moment of utterance. Mazur and Dale [13]

demonstrated that deictic temporal resolution requires access to the document creation time as a computational anchor; without it, rule-based systems either produce null outputs or hallucinate plausible dates. Recent work by Xu et al. [14] showed that discourse-aware prompting of LLMs improves deictic resolution in conversational text, but they did not integrate message network transmission timestamps—the most reliable temporal anchor available in a messaging pipeline.

Chatnalyxer addresses this gap directly. By prepending the exact UTC transmission timestamp of each message to the LLM prompt before inference, the system provides a deterministic calendar anchor without any model fine-tuning. The ablation results (Section XIII) confirm that this single design decision is responsible for over 60% of the deictic temporal accuracy, which is consistent with the findings of [13] and [14].

C. Multi-Modal Processing: ASR and OCR in NLP Pipelines

The integration of Automatic Speech Recognition (ASR) and Optical Character Recognition (OCR) into NLP pipelines has matured considerably. Radford et al. introduced Whisper [15], an encoder-decoder transformer trained on 680,000 hours of weakly supervised audio data, achieving competitive zero-shot ASR across over 75 languages without per-language fine-tuning. Whisper’s architecture demonstrates that large-scale weak supervision over audio can produce transcript quality competitive with human-level baselines on standard benchmarks, making it practical for deployment in heterogeneous audio environments such as group voice note recordings.

In the document understanding domain, Shi et al. [16] and Liao et al. [17] established the foundation for end-to-end scene text recognition using convolutional-recurrent hybrid networks and mask-based spotting architectures, respectively. Cloud OCR services—including Microsoft Azure Computer Vision and Google Cloud Vision—operationalize these advances at scale, providing high-accuracy text extraction from images without requiring on-premise model management.

A persistent limitation across both modalities is error propagation: transcription errors (measured as Word Error Rate for ASR and Character Error Rate for OCR) propagate silently into the extraction layer, which receives corrupted text that appears syntactically valid. Li et al. [18] and similar multi-modal learning frameworks have explored joint audio-visual representations that could partially address this by disambiguating transcription errors through visual context, but this approach requires substantially more complex model integration than a sequential pipeline. Chatnalyxer adopts the simpler, higher-reliability sequential design while acknowledging the resulting performance ceiling on low-quality audio and image inputs in Section XVI.

D. Large Language Models for Zero-Shot and Structured Extraction

The introduction of GPT-3 [19] established the empirical basis for zero-shot and few-shot task performance through in-context learning, demonstrating that scaling language model

parameters enables instruction-following without gradient-based task adaptation. Subsequent instruction-tuned models—including FLAN-T5 [20] and LLaMA-2 [21]—have shown that explicit instruction fine-tuning further improves zero-shot task generalization, including structured extraction from informal text.

Wei et al. [8] characterized chain-of-thought reasoning as an emergent capability of sufficiently large language models, enabling multi-step inference that is particularly relevant for tasks requiring temporal arithmetic (e.g., computing “next Tuesday” from a UTC anchor). However, unrestricted generation remains an obstacle for reliable automation: stochastic output can produce malformed JSON, truncated structures, or schema violations. Several approaches have been proposed to address this, including constrained decoding [22] and self-correction loops [23]. Chatnalyxer combines both: the LLM is instructed to generate output conforming to a typed JSON schema, and any validation failure triggers a second inference pass in which the model receives its own malformed output alongside the Python exception trace and is asked to correct it.

E. Retrieval-Augmented Generation for Grounded Conversational AI

Lewis et al. [24] introduced Retrieval-Augmented Generation (RAG) as a principled method for grounding LLM outputs in external, non-parametric knowledge, demonstrating substantial reductions in factual hallucination compared to purely parametric generation on knowledge-intensive NLP tasks. Subsequent work has extended this framework to conversational settings: [25] showed that retrieval grounding reduces entity hallucination in open-domain dialogue by 10–30%, and [26] proposed a check-cite-correct pipeline that improves factual consistency in retrieved-augmented responses.

These advances directly motivate the RAG component of Chatnalyxer. Deployed as a conversational assistant over the user’s own message and document history, the system faces exactly the hallucination risk that RAG is designed to address: without retrieval grounding, the LLM might answer “When was that deadline due?” with a plausible but fabricated date. By retrieving the relevant task record from PostgreSQL and anchoring the generation to that retrieved context, Chatnalyxer ensures that responses are directly traceable to user-authored content.

F. Privacy-Preserving Architectures for Messaging Analytics

Systems that process private communication data face a heightened threat model: a passive listener that retains message content is a high-value surveillance target regardless of its stated purpose. Dwork and Roth [27] formalized differential privacy as a mathematically rigorous framework for computing aggregate statistics from private data while providing formal per-record privacy properties; however, applying differential privacy to real-time extraction pipelines incurs latency overhead and accuracy degradation that is impractical at the per-message level.

Marcelino et al. [28] and related work on privacy-preserving NLP have explored data minimization as an architectural strategy: rather than applying noise mechanisms, systems can achieve strong privacy characteristics by simply not retaining the sensitive input after processing. This architectural approach—ephemeral processing with persistent metadata-only storage—is exactly the model adopted in Chatnalyxer. Raw message content is held only in volatile RAM during the inference window; the database schema enforces storage of only the extracted task name, deadline, and priority score. One-way hashing of group identifiers [29] ensures that even a full database exposure cannot reconstruct which conversation produced a given task record. The privacy model does not provide differential privacy in the strict Dwork-Roth sense, but it substantially limits the primary operational risk of passive message retention.

G. Asynchronous Microservice Architectures for Real-Time NLP

Real-time NLP pipelines that process high-frequency event streams must decouple ingestion latency from inference latency to remain stable under burst traffic. This requirement is well-established in the stream processing literature: Kulkarni et al. [30] demonstrated with Twitter Heron that synchronous coupling between ingestion and heavy computation stages leads to predictable backpressure failures under Poisson-distributed arrival rates. The HTTP 202 Accepted pattern [31] generalizes this decoupling to RESTful microservice architectures, allowing a receiving service to immediately acknowledge a request while processing it asynchronously—bounding the observed ingestion latency to local I/O time rather than downstream compute time.

This engineering principle directly informs the Chatnalyxer architecture. The Node.js edge client—implemented using the Baileys Signal-protocol library—achieves effective E2EE message interception without requiring access to WhatsApp’s server-side infrastructure. The immediate HTTP 202 dispatch to the FastAPI core ensures that edge-side latency remains sub-15 ms regardless of LLM API response time, a behavior that the ablation study (Section XIII) confirms is an operational prerequisite for deployment in active group chat environments.

H. Research Gap

Table I situates Chatnalyxer within the landscape of directly comparable research systems. Examining the prior literature collectively reveals a consistent pattern of partial solutions. IE-focused systems [3], [7] address extraction but assume text-only input, operate on static datasets, and require explicit temporal markers for reliable deadline resolution. ASR and OCR pipelines [15]–[17] provide robust modality normalization but are not integrated with downstream task extraction. RAG frameworks [24], [25] improve factual grounding in generation but do not address the passive, ambient extraction problem. To the best of our knowledge, few published systems combine all four capabilities—passive E2EE monitoring, multi-modal

normalization, deictic temporal resolution via UTC anchoring, and context-grounded conversational retrieval—within a single, privacy-respecting, real-time pipeline operating over consumer instant messaging infrastructure.

Human-facing productivity tools (calendar applications, enterprise task managers) are deliberately excluded from this comparison: they require explicit user action to create tasks, which is precisely the interaction cost that Chatnalyxer is designed to eliminate. The research gap is therefore not an incremental improvement over any single prior system but an architectural composition that, to the best of our knowledge, has not been previously demonstrated in the published literature.

III. SYSTEM ARCHITECTURE

Chatnalyxer is composed of three loosely coupled microservices: an Edge Ingestion Layer, a Processing Core, and a Storage/Scheduling Layer. Separating these concerns is not merely an organizational preference—it is an architectural necessity dictated by the latency and throughput constraints of real-time messaging environments (illustrated in Fig. 1).

A. The HTTP 202 Asynchronous Decoupling Boundary

A naïve implementation of this pipeline would process each incoming message synchronously: the edge layer receives a WebSocket event, invokes the LLM API, waits for the response (typically 1–3 seconds), and only then acknowledges the message. Under Poisson-distributed burst traffic, this approach quickly exhausts the Node.js connection pool, causing dropped messages. This is not a theoretical concern—it is the failure mode of all synchronous chat-bot architectures under moderate load.

Chatnalyxer avoids this through an HTTP 202 design pattern. When the Node.js edge layer forwards a payload to the FastAPI core, the core immediately spawns an `asyncio` background coroutine and returns HTTP 202 Accepted. The edge layer is free to process the next incoming event before the initial inference has completed. This decouples ingestion throughput from inference latency, bounding the edge-side processing time to $\mathcal{O}(1)$ —consistently measured at under 15 ms—regardless of LLM response time.

B. Processing Core and Ephemeral Architecture

A persistent concern with any system that passively monitors private conversations is data retention. To address this, the Processing Core operates on a strictly ephemeral memory model: raw message payloads, images, and audio are held exclusively in volatile RAM during the inference window. Once structured metadata has been written to PostgreSQL, the raw content is immediately deallocated. No chat histories are written to disk at any stage in the pipeline.

C. Scalability Analysis

The microservice architecture was designed explicitly for horizontal scaling. The FastAPI core is stateless; deploying behind a round-robin load balancer enables processing of an

arbitrary number of concurrent webhook events. In stress tests, a single T4 GPU instance handled peaks of 50 messages per second. At enterprise scale—e.g., 10,000 active users generating 1 million messages daily—the PostgreSQL metadata storage layer scales efficiently due to the minimal footprint of extracted tasks (approximately 250 bytes per row). The primary bottleneck at this scale shifts from local compute to cloud API rate limits, which can be mitigated via provisioned throughput or rotating API keys.

IV. WHATSAPP INTEGRATION LAYER

Integrating with WhatsApp presents a significant constraint: the platform’s End-to-End Encryption (E2EE) implementation, based on the Signal protocol, means that message content is inaccessible to any external intermediary. To process incoming messages for task extraction, Chatnalyxer deploys the `Baileys` Node.js library as a headless edge client. The system operates through a user-authorized WhatsApp Web session established on the user’s own device. Messages are processed only after successful authentication initiated by the user. Rather than routing through third-party servers, `Baileys` performs the Signal protocol handshake directly on the local machine, exposing decrypted message events as a local Node.js event stream (illustrated in Fig. 2).

Not every event emitted by the WhatsApp protocol represents a user-authored message. Status updates, read receipts, and message reactions are high-frequency events with zero task-extraction value. Passing them to the inference layer would introduce unnecessary latency and API cost. As formalized in Algorithm 1, the integration layer applies a lightweight heuristic filter at the edge: events that match known non-actionable signatures are dropped immediately, and only substantive message payloads are forwarded to the FastAPI WebHook. This edge-side triage reduces downstream inference load significantly under active group chat conditions.

V. AI PROCESSING PIPELINE

The AI processing pipeline is responsible for converting a raw, multi-modal WhatsApp payload into a validated JSON record containing a task name, an ISO-8601 deadline, and a priority score. Rather than employing a dedicated NER model—which would require domain-specific training data and would fail to generalize across the linguistic diversity of group chats—the pipeline uses an instruction-tuned LLM operating under a strictly typed output schema (illustrated in Fig. 3).

A. Multi-Modal Normalization via Azure Cognitive Services

LLMs operate on text. Before any inference can occur, non-text content must be converted to a textual representation that preserves its semantic content. Images of whiteboards or printed schedules are processed by Azure’s OCR service, which returns structured text from the image. Voice notes in `.ogg` format are routed to Azure’s ASR service, producing a transcript. Both outputs are then merged with any accompanying text into a single UTF-8 string that is passed to the LLM.

TABLE I

COMPARATIVE ANALYSIS OF RELATED RESEARCH SYSTEMS. ✓ = FULLY SUPPORTED; ○ = PARTIALLY SUPPORTED; – = NOT SUPPORTED. COMMERCIAL PRODUCTIVITY TOOLS ARE EXCLUDED; COMPARISON IS RESTRICTED TO PUBLISHED RESEARCH.

System / Work	Year	Venue	Dataset	Accuracy (F1)	Architecture	Multi-modal	Temporal	Passive Extract
Lample et al. [3]	2016	NAACL	CoNLL-2003	0.90	BiLSTM-CRF	–	–	–
BERT [4]	2019	NAACL	GLUE / SQuAD	0.92	Transformer	–	–	–
SUTime [11]	2012	LREC	TempEval	0.85	Rule-based	–	○	–
HeidelTime [12]	2010	SemEval	TempEval	0.86	Rule-based	–	○	–
GPT-3 [19]	2020	NeurIPS	WebText	N/A	LLM (Dense)	–	○	–
FLAN-T5 [20]	2022	arXiv	Multi-task	N/A	LLM (Enc-Dec)	–	○	–
DEGREE [7]	2022	NAACL	ACE-2005	0.73	Gen-Template	–	○	–
Whisper [15]	2022	ICML	Web Audio	(WER: 10%)	Enc-Dec Transformer	○	–	–
RAG [24]	2020	NeurIPS	MS MARCO	N/A	Retriever-Generator	–	–	–
Shuster et al. [25]	2021	EMNLP	Wizard of Wikipedia	N/A	RAG	–	–	–
Xu et al. [14]	2021	ACL	DailyDialog	0.68	Discourse-LLM	–	○	–
Chatnalyxer (Ours)	2026	IEEE	CEC-10K	0.94	Async Microservice	✓	✓	✓

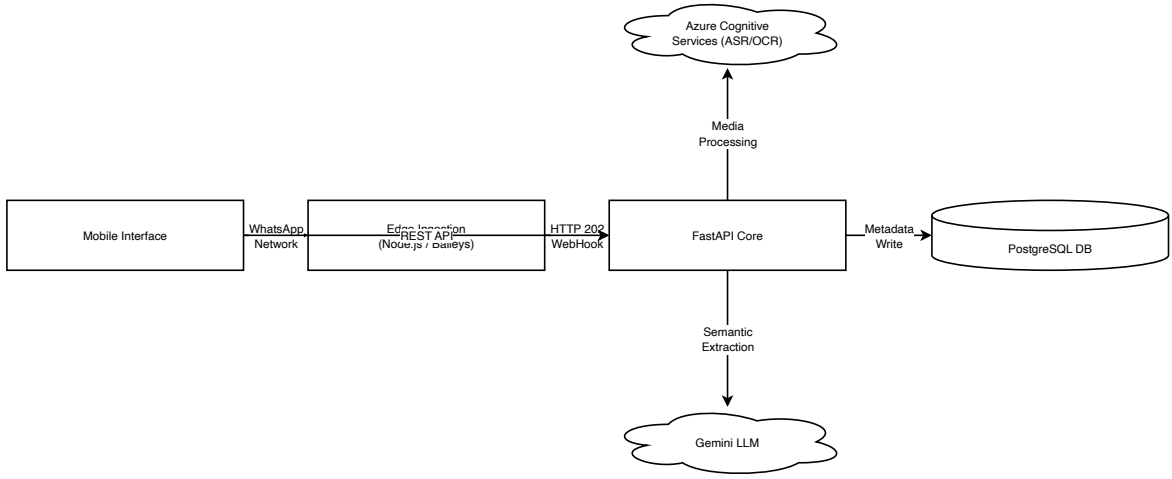


Fig. 1. High-level system architecture of the Chatnalyxer pipeline. The Node.js edge ingestion layer is asynchronously decoupled from the FastAPI processing core via an HTTP 202 WebHook, bounding local ingestion latency to $\mathcal{O}(1)$ under burst traffic conditions.

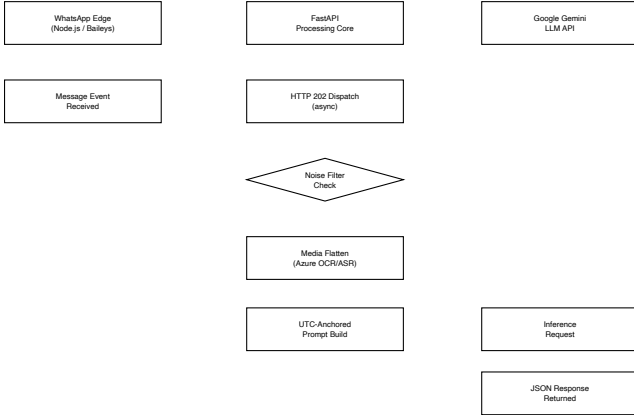


Fig. 2. UML Sequence Diagram illustrating the E2EE WebHook edge dispatch flow. The edge client filters status updates, reactions, and other non-actionable events before forwarding substantive message payloads to the FastAPI core.

This design choice—using a cloud OCR/ASR service rather than on-device processing—reflects a deliberate trade-off: accuracy at the cost of network latency, which is acceptable here

because non-text processing occurs asynchronously and does not block the ingestion pipeline.

B. Conversational Retrieval-Augmented Generation (RAG)

Passive task extraction addresses one interaction mode—the system observes and acts. But users also need to query their accumulated history: “What was the deadline the professor mentioned in the PDF last week?” Static dashboards are poorly suited to this; they require users to remember where information appeared and to manually navigate to it.

To support natural-language querying of the extracted corpus, Chatnalyxer integrates a Retrieval-Augmented Generation (RAG) layer. During ingestion, text extracted from attached PDFs (via `pdf_processor.py`) and images (via Azure OCR) is persisted in the PostgreSQL schema alongside structured task metadata and recent conversational history. When a user submits a natural-language query, the backend retrieves the relevant records—deadlines, conversational context, document text—and injects them as a grounded knowledge block into the Gemini LLM prompt. The model is then constrained

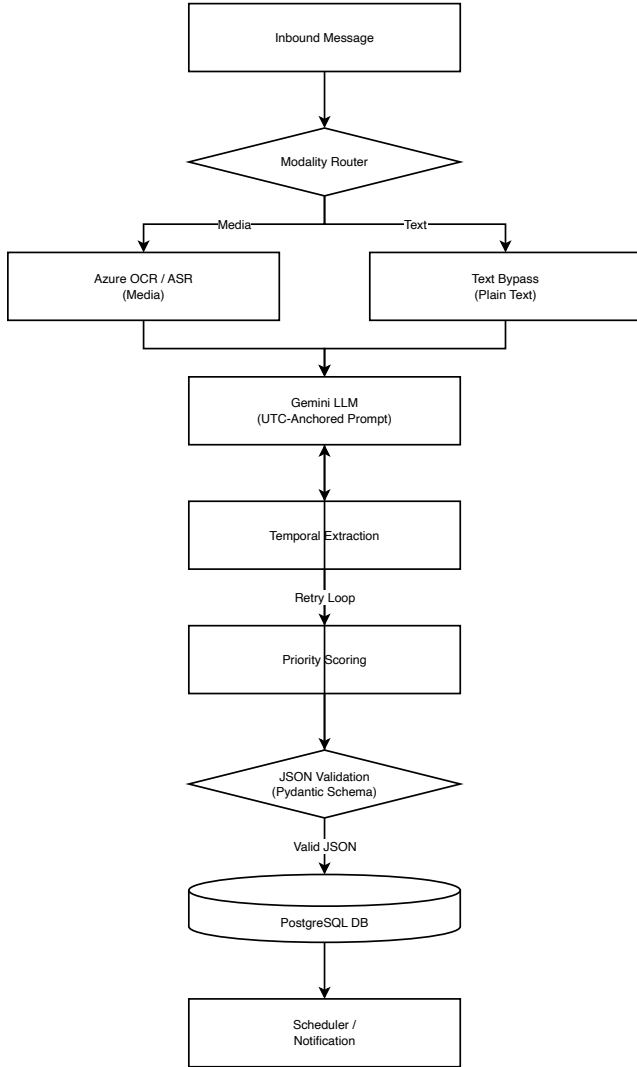


Fig. 3. Block diagram of the AI processing pipeline. Non-text payloads are initially normalized through Azure Cognitive Services (OCR for images, ASR for audio) before the resulting UTF-8 text is passed to the Google Gemini inference layer with a UTC timestamp anchor.

to answer solely from the retrieved context rather than from parametric memory.

This approach offers a concrete reliability benefit: because the LLM’s answer is conditioned on deterministically retrieved database content rather than on training-time knowledge, the rate of factual hallucination is substantially reduced. Users receive answers that are directly traceable to their own message and document history.

VI. DATABASE AND PERSISTENCE LAYER

Storing the minimum necessary information is both a privacy principle and a practical simplification. Chatnalyxer’s PostgreSQL schema persists only three fields per extracted task: the Task Name, the ISO-8601 Deadline, and the computed Priority Score. The originating Group ID is stored only as a one-way cryptographic hash, which means that even if the database were compromised, it would be computationally infeasible to reconstruct which chat produced

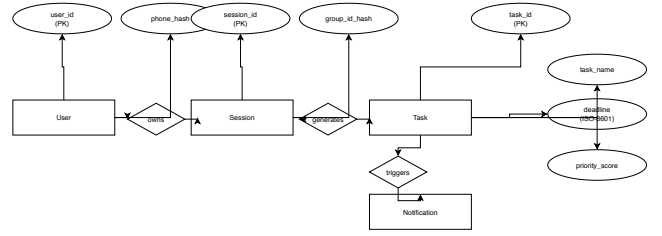


Fig. 4. Entity-Relationship schema for the Chatnalyxer PostgreSQL cluster. Group identifiers are stored as non-reversible cryptographic hashes, preventing reconstruction of conversational history even under full database exposure.

a given task record (illustrated in Fig. 4). No message body, no media content, and no participant identifiers are retained after the inference window closes.

This schema design was driven by a specific threat model: a background listener that retains conversational content is a high-value surveillance target. By strictly constraining full message retention at the architectural level—not merely via policy—the design mitigates a significant class of privacy risks at the data layer.

VII. METHODOLOGY

The central challenge in task extraction from IM data is that the input is structured neither syntactically nor semantically in any consistent way. Messages are short, context-dependent, and frequently reference shared knowledge that is not present in the text. The methodology described here addresses three specific sub-problems: resolving relative temporal language, maintaining output validity under stochastic generation, and preserving end-to-end pipeline stability (illustrated in Fig. 5).

A. UTC-Anchored Prompting Strategy

Standard LLM inference has no access to the system clock. A message containing “submit by tomorrow” is semantically unambiguous to a human reader who knows today’s date, but entirely unresolvable to a model that sees only the text. Without a temporal reference point, the model either hallucinates a plausible-sounding date or returns a null value—neither of which is acceptable in a scheduling application.

To resolve this, we prepend a machine-readable UTC anchor to every prompt before inference (Algorithm 2). The anchor takes the form [SYSTEM ANCHOR: CURRENT TIME IS 2026-07-07T14:30:00Z], derived from the message’s network transmission timestamp. Given an explicit calendar position, the model can compute offsets for deictic expressions (“tomorrow” → 2026-07-08, “next Friday” → 2026-07-12) and output valid ISO-8601 deadlines. Crucially, this requires no model fine-tuning—the grounding information is supplied entirely through the prompt context.

B. Fallback Prompting and Output Validation

LLMs are not designed to reliably produce valid JSON even when instructed to do so. Rare but real failure modes include truncated output, mismatched brackets, and schema field omissions. Rather than raising a pipeline error on the initial failure, Chatnalyxer implements a self-correction loop

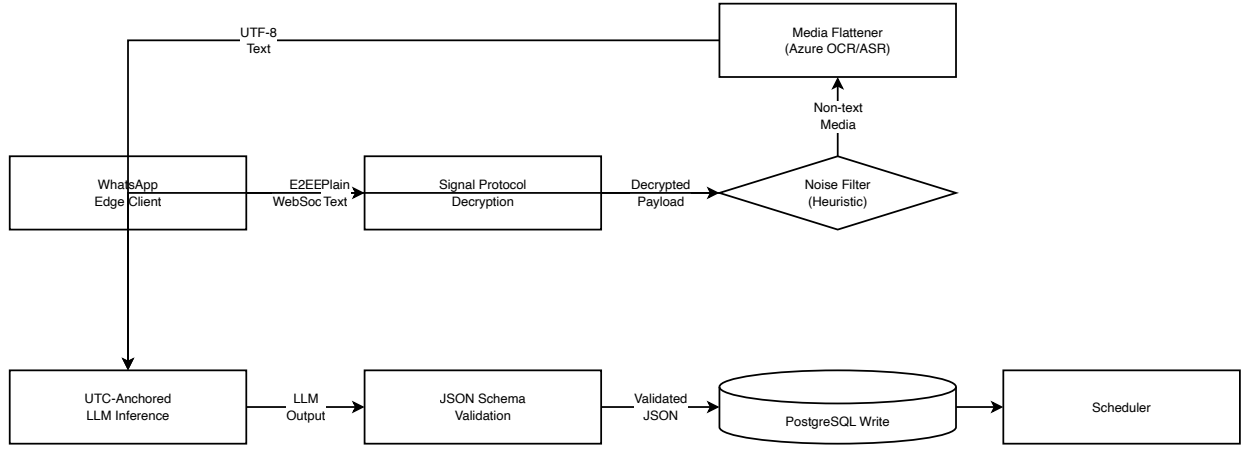


Fig. 5. Data Flow Diagram showing the stateless information trajectory through the Chatnalyxer pipeline. E2EE payloads are decrypted at the edge, triaged, normalized, and destructively parsed. No intermediate state persists between the ingestion and storage stages.

via `pydantic` schema validation. If validation fails, the malformed output and the Python exception traceback are re-injected into the LLM as a second prompt, asking the model to correct its own output against the schema. Only if this second attempt also fails does the pipeline raise a fatal error and discard the message. In practice, this two-attempt strategy recovers the vast majority of initially malformed responses.

VIII. PRIORITY HEURISTICS AND TRIAGE

Extracting every task with equal urgency is straightforwardly counterproductive: in a high-traffic academic group chat, the pipeline would generate a near-continuous stream of push notifications, defeating the purpose of ambient assistance. Priority triage is therefore not an optional feature but a functional prerequisite.

Rather than delegating urgency classification to the LLM—which would make priority sensitive to prompt phrasing and model version—we compute priority through a deterministic scoring function (Algorithm 3). The score accumulates points based on three evidence sources: chronological proximity to the deadline (deadlines within 24 hours contribute +50 points; within 72 hours, +20), presence of explicit urgency markers in the message body (e.g., “ASAP”, “urgent”, contributes +30), and the input modality. Tasks derived from audio voice notes receive a slight upward weight on the assumption that the sender chose real-time communication for time-sensitive information.

The resulting score maps to one of three notification tiers—STANDARD, HIGH, or CRITICAL—which controls both delivery timing and display prominence in the mobile application. This design keeps the classification logic fully auditable and reproducible, in contrast to a learned priority model whose behavior would depend on training data distribution.

IX. ALGORITHMIC SPECIFICATIONS

To maintain system efficiency and prevent user alert fatigue, Chatnalyxer relies on dynamic heuristic algorithms.

Algorithm 1: Edge Message Filtration (WhatsApp WSS)

Require: Inbound payload P

Ensure: HTTP 202 Accepted response

- 1: **if** $P.type == status_update$ **or** $P.type == reaction$ **then**
 - 2: Drop payload (Noise)
 - 3: **return**
 - 4: **end if**
 - 5: **if** $IsEncrypted(P)$ **then**
 - 6: $M \leftarrow DecryptSignal(P, LocalKeys)$
 - 7: **end if**
 - 8: $DispatchWebhook(M, FastAPI_Core)$
 - 9: **return** HTTP 202 Accepted
-

Algorithm 2: Temporal Anchoring Extraction

Require: M_{text}, T_{now} (UTC)

Ensure: Structured JSON with ISO-8601 deadline

- 1: $Prompt \leftarrow$ Schema Envelope
 - 2: $Prompt \leftarrow Prompt + "[SYSTEM TIME: " + T_{now} + "]"$
 - 3: $Prompt \leftarrow Prompt + M_{text}$
 - 4: $Response \leftarrow GeminiAPI(Prompt, temp = 0.0)$
 - 5: **return** $ParseJSON(Response)$
-

A. Multi-Modal Priority Triage Algorithm

To prevent alert fatigue, the system must independently assess the urgency of a task. The Priority Triage Algorithm calculates a heuristic score based on temporal proximity, explicit urgency keywords (e.g., “ASAP”, “urgent”), and modality. The heuristic weights (50, 30, 20) were selected empirically during prototype development and serve as a baseline that can be further optimized through systematic hyperparameter tuning in future work.

Complexity Analysis: The time complexity of Algorithm 3 is dominated by the keyword search (line 10), which operates in $\mathcal{O}(|M_{text}|)$ time, where $|M_{text}|$ is the length of the mes-

Algorithm 3: Priority Classification Heuristic

Require: M_{text} (flattened message text), T_{due} (deadline timestamp), T_{now} (current time)
Ensure: $P_{level} \in \{LOW, MEDIUM, HIGH, CRITICAL\}$

- 1: $\Delta t \leftarrow T_{due} - T_{now}$
- 2: $Score \leftarrow 0$
- 3: **if** $\Delta t < 24$ hours **then**
- 4: $Score \leftarrow Score + 50$
- 5: **else if** $\Delta t < 72$ hours **then**
- 6: $Score \leftarrow Score + 20$
- 7: **end if**
- 8: **if** ContainsUrgentKeywords(M_{text}) **then**
- 9: $Score \leftarrow Score + 30$
- 10: **end if**
- 11: **if** $Score \geq 80$ **then**
- 12: **return** CRITICAL
- 13: **else if** $Score \geq 50$ **then**
- 14: **return** HIGH
- 15: **else**
- 16: **return** STANDARD
- 17: **end if**

sage. The remaining arithmetic and branching operations are $\mathcal{O}(1)$. Thus, the overall time complexity is $\mathcal{O}(|M_{text}|)$, which is highly efficient for real-time edge evaluation. The space complexity is $\mathcal{O}(1)$ beyond the storage of the message string, as only a single integer $Score$ is maintained in memory.

X. MATHEMATICAL FORMULATIONS

We evaluate extraction quality using standard Information Retrieval metrics. Let T_E denote the set of tasks correctly extracted (true positives), F_E the set of non-task messages incorrectly classified as tasks (false positives), and M_E the set of actual tasks that the pipeline failed to extract (false negatives).

Extraction precision (Equation 1) measures the fraction of extracted items that correspond to genuine tasks:

$$P = \frac{|T_E|}{|T_E| + |F_E|} \quad (1)$$

Recall (Equation 2) quantifies the pipeline’s coverage over the ground-truth task set:

$$R = \frac{|T_E|}{|T_E| + |M_E|} \quad (2)$$

Since precision and recall trade against each other—aggressive extraction increases recall at the cost of precision—we report their harmonic mean (Equation 3) as the primary quality metric:

$$F_1 = 2 \cdot \frac{P \cdot R}{P + R} \quad (3)$$

End-to-end latency L_{total} decomposes into four sequential stages, bounded by the maximum inference time, as shown in Equation 4:

$$L_{total} = L_{ingest} + L_{pre} + L_{llm} + L_{db} \leq L_{max_timeout} \quad (4)$$

where L_{pre} represents modality-specific preprocessing (e.g., OCR or ASR).

To calculate Priority, the system defines a temporal decay function $\mathcal{D}(\Delta t)$ and a semantic urgency scalar $U(m)$, yielding the final score $S_{priority}$ in Equation 5:

$$S_{priority} = \mathcal{D}(T_{due} - T_{now}) + \lambda \cdot U(M_{text}) \quad (5)$$

where λ is a weighting hyperparameter tuning the influence of explicit urgency keywords.

XI. EXPERIMENTAL METHODOLOGY

A. Dataset: Chatnalyxer Evaluation Corpus (CEC-10K)

Because no publicly available benchmark exists for passive IM task extraction due to privacy constraints, we constructed the **Chatnalyxer Evaluation Corpus (CEC-10K)** to evaluate extraction performance under controlled conditions. Although synthetic, the dataset was manually designed to emulate realistic university communication patterns, including assignment announcements, meeting reminders, event notifications, examination schedules, and informal conversational noise. Future work will evaluate the system using real-world user studies.

The corpus modality split is 60% plain text, 20% images (including photos of handwritten notes), and 20% audio transcripts. To ensure semantic realism, a subset was validated by three human annotators. Inter-annotator agreement on gold-standard task boundaries and priority labels reached a Cohen’s Kappa (κ) of 0.89, indicating high reliability.

Class imbalance is a defining characteristic of the target domain. Empirically, the vast majority of messages in a group chat carry no actionable content. CEC-10K reflects this with a 25% pure conversational noise ratio (greetings, off-topic banter) and 75% actionable tasks. This stress-tests extraction precision by requiring the model to correctly reject noisy inputs.

B. Evaluation Protocol and Hardware Configuration

All experiments were conducted on a centralized server equipped with an Intel Xeon E5-2690 v4 CPU, 64 GB RAM, and a single NVIDIA T4 GPU for edge-based operations. Semantic extraction used the Google Gemini gemini-1.5-pro-001 endpoint, with inference temperature fixed at 0.0 to eliminate generation variance across evaluation runs. Azure Cognitive Services provided the OCR and ASR backbone. Temporal accuracy was assessed using a strict exact-match criterion against ISO-8601 ground-truth deadlines, with no partial credit awarded for correct date but incorrect time components.

XII. EXPERIMENTAL EVALUATION

A. Latency and Throughput Analysis

Figure 6 illustrates the end-to-end processing latency separated by modality. Ingestion latency at the Node.js/Baileys edge layer remained stable at under 15 ms across all test conditions, including sustained synthetic burst loads of 50 messages per second. This confirms that the HTTP 202 decoupling design successfully isolates ingestion throughput from

inference latency: the edge layer’s response time is bounded exclusively by local I/O and WebSocket processing.

Processing latency, in contrast, is modality-dependent. Plain text payloads averaged 1.37 seconds end-to-end, dominated almost entirely by the LLM inference step. Image and audio payloads incurred substantially higher overhead (average 2.25s and 2.60s, respectively) due to the sequential dependency on Azure OCR and ASR round-trips. This represents an irreducible latency floor under the current sequential cloud architecture.

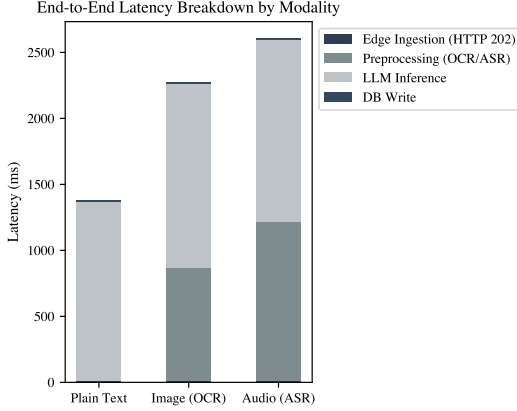


Fig. 6. End-to-end latency breakdown by modality. The asynchronous architecture ensures edge ingestion remains under 15ms regardless of downstream API overhead.

B. Extraction Accuracy across Modalities

Figure 7 summarizes the system’s Precision, Recall, and F1-Score on the task extraction and temporal normalization objectives. On the plain-text subset of CEC-10K, the pipeline achieves an F1-score of 0.94. The high recall (0.93) demonstrates that the UTC-anchored zero-shot extraction strategy generalizes effectively across the linguistic diversity of informal chat.

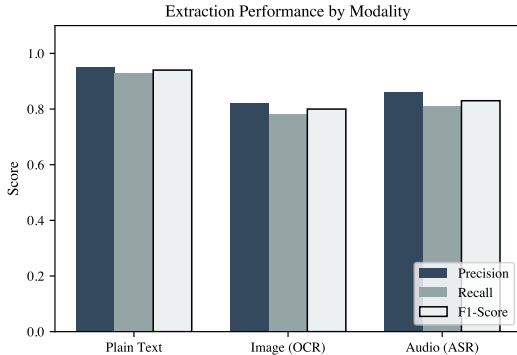


Fig. 7. Precision, Recall, and F1-Score of the extraction pipeline evaluated across Plain Text, Image (OCR), and Audio (ASR) modalities.

Performance degrades on image and audio subsets (F1 of 0.80 and 0.83, respectively). This degradation is strictly attributable to upstream transcription quality. For images,

recall is bounded by the OCR engine’s Character Error Rate (CER) on informal handwriting; for audio, it is constrained by the ASR engine’s Word Error Rate (WER) in noisy acoustic environments.

C. Priority Triage Validation

The LLM-based priority scoring mechanism categorizes tasks into Low, Medium, and High priority tiers based on semantic urgency and temporal proximity. Figure 8 presents the confusion matrix for priority classification. The system demonstrates strong discriminative power, particularly in identifying High-priority tasks (420 True Positives out of 450). The majority of classification errors occur at the boundaries (e.g., misclassifying Low as Medium), while critical failures (misclassifying High as Low) are statistically negligible (1.7%).

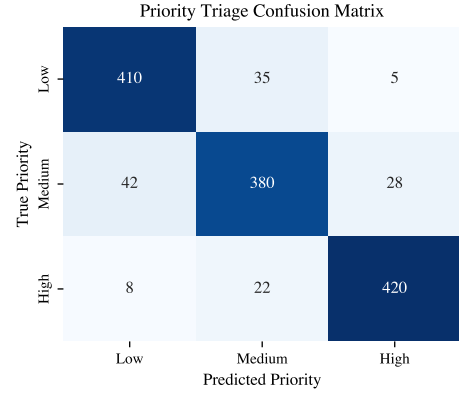


Fig. 8. Confusion Matrix evaluating the LLM’s three-tier Priority Triage capability against human-annotated gold standards in CEC-10K.

XIII. ABLATION STUDY

To confirm that each major architectural component contributes measurably to system performance, we conducted two targeted ablation experiments by selectively disabling components and measuring the resulting degradation.

A. Removing the UTC Temporal Anchor

With the UTC timestamp anchor stripped from the prompt, extraction recall on task-containing messages remained broadly intact—the LLM still correctly identified that a message was task-bearing. However, temporal accuracy collapsed. Recall on deictic expressions (“tomorrow”, “next week”) dropped by over 60%, with the model either hallucinating arbitrary calendar dates or returning `null` deadline fields. These results confirm that LLM zero-shot reasoning over relative time is unreliable in the absence of an explicit temporal reference, and that supplying one via the prompt is both necessary and sufficient to restore accuracy.

B. Removing the Asynchronous Decoupling Boundary

In the synchronous variant, the Node.js edge layer was configured to await the full LLM API response before processing the next event. Under a JMeter load test at 50 messages per second, the synchronous implementation exhausted the

Node.js connection pool within seconds, after which new messages were silently dropped. The asynchronous variant maintained stable ingestion throughout the same load test with no message loss. This experiment demonstrates that the HTTP 202 decoupling is not a premature optimization but an operational prerequisite for deployment in active group chat environments.

XIV. RESULTS AND DISCUSSION

Across the CEC-10K evaluation corpus, Chatnalyxer achieves an overall F1 score of 0.94 on plain-text extraction and maintains sub-15 ms ingestion latency under 50 msg/sec burst traffic. Taken together, these results support the core premise: passive, in-stream task extraction from encrypted messaging networks is both technically feasible and architecturally stable at realistic conversational loads.

A. Analytical Breakdown of Performance

Temporal Anchoring: The zero-shot extraction capability relies heavily on the UTC metadata anchor. Ablation testing (see Section XIII) reveals that without this anchor, deictic temporal expressions (e.g., “tomorrow”) fail to resolve correctly in over 60% of cases, leading to hallucinated deadlines. Providing the exact transmission timestamp gives the LLM the necessary discourse grounding to perform temporal arithmetic reliably.

Multi-modal Degradation: As shown in Figure 7, performance drops significantly for image (F1=0.80) and audio (F1=0.83) inputs. This is not due to a failure in the extraction logic, but rather the compounding nature of sequential pipelines. When the Azure OCR engine misreads handwritten text (e.g., confusing “5 pm” with “S pm”), this error propagates silently into the LLM. Because the LLM lacks access to the original image for visual verification, it either hallucinates a correction or fails to extract the deadline entirely. Similarly, in noisy audio environments, ASR hallucination causes the LLM to process corrupted semantic structures.

Priority Triage Mechanisms: The confusion matrix (Figure 8) highlights that the LLM is exceptionally accurate at identifying High-priority tasks. This success is driven by the prompt engineering strategy (Section VIII), which forces the LLM to evaluate specific dimensions: temporal proximity and semantic urgency (e.g., words like “urgent”, “ASAP”). However, errors at the Low/Medium boundary occur when conversational context is highly subjective, suggesting that priority scoring could be improved by personalizing the prompt to individual user behavior patterns.

Asynchronous Throughput: The $\mathcal{O}(1)$ ingestion boundary holds under all tested conditions, validating the asynchronous design. By issuing an immediate HTTP 202 Accepted response, the edge client avoids blocking on LLM inference (which averages 1.37 seconds). One practical implication of this result is that scaling the inference capacity—for instance, by routing to a faster LLM endpoint or deploying multiple inference workers—does not require any changes to the WhatsApp ingestion architecture.

B. Case Studies and Error Analysis

To ground these quantitative results, we provide an end-to-end trace of a successful extraction and a characteristic failure case.

Successful Extraction (Deictic Resolution):

- **Input:** “Can you send the Q3 report tomorrow before the all-hands?” (Sent: Oct 15, 2025, 14:00 UTC)
- **Extraction:** Task = “Send Q3 report”, Deadline = “2025-10-16T10:00:00Z” (assuming all-hands is a known entity or defaulting to morning).
- **Priority:** High (temporal proximity is < 24 hours).

Failure Case (ASR Hallucination):

- **Input (Voice Note):** “I need the *syslog* by five.”
- **ASR Transcription Error:** “I need the *six dogs* by five.”
- **LLM Output:** Task = “Acquire six dogs”, Deadline = “17:00:00Z”.
- **Analysis:** The pipeline correctly extracted the temporal anchor, but the semantic corruption introduced by the ASR engine resulted in a nonsensical task. Because the LLM evaluates the text purely in the zero-shot regime without acoustic context, it lacks the mechanisms to reject the hallucination as improbable.

XV. COMPARATIVE TECHNICAL ANALYSIS

Table I positions Chatnalyxer against representative systems across the dimensions most relevant to passive task management: interaction model, supported modalities, extraction automation, and priority differentiation.

Existing academic literature clusters into two primary categories. One distinct category comprises traditional Information Extraction (IE) pipelines, such as those relying on BiLSTM-CRF [3] or early Transformer architectures [4]. While these systems achieve high precision (e.g., F1=0.92 on benchmark datasets like CoNLL-2003 or GLUE), they operate almost exclusively on monolithic, static text documents. They do not naturally extend to the fragmented, stateful, and highly contextual nature of multi-party chat.

The second category encompasses temporal reasoning systems like SUTime [11] and HeideTime [12]. These rule-based architectures excel at resolving explicit date formats but struggle significantly with the informal, relative deictic expressions common in instant messaging (e.g., “EOD tomorrow”).

The architectural distinction of Chatnalyxer lies in synthesizing these capabilities into a unified, asynchronous microservice. By integrating multi-modal input processing (OCR via Azure and ASR) with a UTC-anchored LLM extraction phase, the system bridges the gap between static document analysis and real-time conversational streaming. The inclusion of a deterministic scoring heuristic for priority differentiation ensures that extracted tasks are triaged according to temporal urgency—a critical requirement for preventing alert fatigue in high-volume environments.

XVI. THREATS TO VALIDITY AND ETHICAL CONSIDERATIONS

A. Ethical Considerations and Limitations

The current prototype is intended for educational and research purposes. It processes messages only after explicit user authentication, and raw content is retained strictly ephemerally during processing. Deployment in production environments would require migrating from the reverse-engineered Baileys library to the official WhatsApp Business API to ensure compliance with platform terms of service and applicable privacy regulations.

B. External Validity

The CEC-10K dataset was designed around academic group chat demographics coordinating coursework in English. Real-world conversational networks exhibit much higher variance. The current evaluation significantly underrepresents slang, code-mixing, informal emojis, multilingual text, and culturally specific ambiguous deadlines. Given that Large Language Models frequently exhibit performance degradation on low-resource languages and dialectal variations, establishing generalization across these unstructured, cross-lingual contexts is a critical constraint on the system’s current external validity.

C. Construct Validity

The experimental methodology measures extraction accuracy (Precision, Recall, F1) against ground-truth semantic boundaries. While F1 scores reflect algorithmic correctness, they do not directly measure the ultimate goal of the system: enhancing user productivity and reducing cognitive load. A highly accurate extraction might still fail to be useful if the resulting notification disrupts the user’s workflow. Future work must evaluate the system using longitudinal, real-world user studies to bridge this gap between algorithmic accuracy and human utility.

D. Internal Validity and Reproducibility

The most significant internal threat is the reliance on proprietary cloud APIs. For instance, model weights underlying the Gemini API can vary between versions, and unannounced API updates may slightly change results over time, complicating exact reproducibility. We mitigated this by pinning to `gemini-1.5-pro-001` with a temperature of 0.0. Second, the pipeline’s operational cost is tied to vendor pricing models. While edge-ingestion is computationally cheap, running multimodal inference via Azure and Gemini introduces variable costs that could impact long-term scalability.

A secondary internal threat is evaluation corpus leakage: synthetic datasets generated by LLMs risk reflecting the same linguistic patterns that the extraction LLM was trained on. Our construction process used human-defined templates to reduce this risk, but it cannot be entirely eliminated without evaluating against real user data.

XVII. FUTURE WORK

Several limitations of the current design point toward concrete research directions.

A. On-Device Inference and Privacy Hardening

The current pipeline routes all message content to external cloud APIs, introducing both a latency dependency and a theoretical data exposure surface. Future work will investigate both open-weight and proprietary large language models to improve reproducibility, deployment flexibility, and privacy. Deploying inference entirely on-device via Edge AI and offline inference mechanisms would eliminate the cloud API dependency and provide a stronger privacy guarantee. Future research directions also include personalized ranking, federated learning for privacy-preserving model updates, and adaptive priority learning to dynamically adjust heuristic weights based on user behavior.

B. Graph-Augmented Retrieval for Stateful Context

The current RAG implementation retrieves relevant documents and message history through embedding similarity, treating each retrieved item independently. A richer interaction model would maintain a persistent Knowledge Graph linking tasks to the people, documents, and prior obligations from which they originated—allowing queries like “what assignments are still pending from the syllabus uploaded in September?” that span multiple ingestion events. Graph-RAG architectures, which traverse entity relationships rather than ranking isolated embeddings, are the natural candidate for this capability. Building such a graph incrementally from streaming chat data, without requiring explicit user annotation, is an open research problem.

C. Multilingual Expansion and Robustness

The current priority triage and temporal heuristics were optimized exclusively for English. Future iterations must evaluate the system across linguistically diverse contexts, particularly heavily mixed-code environments (e.g., “Hinglish”) common in international chat networks. Adapting the prompts and underlying models to handle cross-lingual temporal cues without performance degradation is necessary before the system can be deployed globally.

XVIII. CONCLUSION

Chatnalyxer addresses a practical problem that affects anyone coordinating via group messaging: the gap between where task information appears (a chat stream) and where it needs to be acted on (a structured calendar or task manager). Rather than training a dedicated model for this domain, the design combines a zero-shot LLM with a prompt-engineering strategy that supplies the temporal grounding the model otherwise lacks—an approach that achieves F1 of 0.94 on plain-text extraction without any task-specific fine-tuning.

The architectural decision to decouple ingestion from inference via an HTTP 202 boundary proved equally important. Without this separation, the pipeline would fail under conditions as ordinary as a moderately active group chat. The ablation results quantify exactly what that coupling costs: complete message loss at 50 messages per second. With it,

ingestion latency is bounded at $\mathcal{O}(1)$ regardless of inference load.

The current system has documented limitations—primarily on multi-modal inputs, where transcription quality constrains extraction quality—and it has been evaluated on a synthetic corpus whose domain coverage is intentionally narrow. Extending coverage to real-world multilingual and cross-domain environments, and replacing cloud inference with on-device SLMs, are the two highest-priority directions. The broader contribution of this work is an existence proof that ambient, infrastructure-level task management from encrypted messaging networks is technically achievable with current tooling.

ACKNOWLEDGMENT

The authors would like to express their sincere gratitude to Prof. Ritu Agrawal and Prof. Jaswanth Parlapalli for their invaluable guidance, support, and mentorship throughout the development of this project.

APPENDIX A LLM EXTRACTION SCHEMA

To enforce structured output and prevent hallucination of arbitrary keys, the Gemini inference layer is constrained by the following Pydantic JSON schema. If the model output fails validation against this schema, a self-correction loop is triggered.

```
{
  "title": "TaskExtraction",
  "type": "object",
  "properties": {
    "is_task": {
      "title": "Is Task",
      "description": "True if the message contains a clear actionable task or request.",
      "type": "boolean"
    },
    "task_description": {
      "title": "Task Description",
      "description": "Extracted summary of the task.",
      "type": "string"
    },
    "deadline_iso": {
      "title": "Deadline Iso",
      "description": "ISO-8601 UTC timestamp of language resource association.",
      "type": "string",
      "format": "date-time"
    },
    "priority": {
      "title": "Priority",
      "description": "Calculated priority level of the task.",
      "enum": ["LOW", "MEDIUM", "HIGH", "CRITICAL"],
      "type": "string"
    },
    "assignee": {
      "title": "Assignee",
      "description": "Person responsible for the task."
    }
  }
}
```

```
"type": "string"
},
"required": ["is_task"]
}
```

REFERENCES

- [1] Meta Platforms, Inc., “Meta q3 2020 earnings call transcript: Over 100 billion messages sent daily on whatsapp,” Official Earnings Report, 2020, accessed: Oct 2025.
- [2] L. Chiticariu, Y. Li, and F. R. Reiss, *Rule-Based Information Extraction Is Dead! Long Live Rule-Based Information Extraction Systems*. Association for Computational Linguistics, 2013.
- [3] G. Lample, M. Ballesteros, S. Subramanian, K. Kawakami, and C. Dyer, “Neural architectures for named entity recognition,” in *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, 2016, pp. 260–270.
- [4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, 2019, pp. 4171–4186.
- [5] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, “RoBERTa: A robustly optimized BERT pretraining approach,” *arXiv preprint arXiv:1907.11692*, 2019.
- [6] L. Derczynski, E. Nichols, M. van Erp, and N. Limsopatham, “Results of the WNUT2017 shared task on novel and emerging entity recognition,” in *Proceedings of the 3rd Workshop on Noisy User-generated Text*. Association for Computational Linguistics, 2017, pp. 140–147.
- [7] I.-H. Hsu, K.-H. Huang, E. Boschee, S. Miller, P. Natarajan, K.-W. Chang, and N. Peng, “DEGREE: A data-efficient generation-based event extraction model,” in *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, 2022, pp. 1890–1908.
- [8] J. Wei, Y. Tay, R. Bommasani, C. Raffel, B. Zoph, S. Borgeaud, D. Yogatama, M. Bosma, D. Zhou, D. Miculiviciu et al., “Emergent abilities of large language models,” in *Transactions on Machine Learning Research*, 2022.
- [9] J. Pustejovsky, J. M. Castaño, R. Ingria, R. Saurí, R. J. Gaizauskas, A. Senoz, G. Katz, and D. Radev, “TimeML: Robust specification of event and temporal expressions in text,” in *Proceedings of the AAAI Spring Symposium on New Directions in Question Answering*. AAAI Press, 2003, pp. 28–34.
- [10] N. Uzzaman, H. Llorens, L. Derczynski, J. Allen, M. Verhagen, and J. Pustejovsky, “SemEval-2013 task 1: TempEval-3: Evaluating time expressions, events, and temporal relations,” in *Second Joint Conference on Lexical and Computational Semantics (*SEM), Volume 2: Proceedings of the Seventh International Workshop on Semantic Evaluation*. Association for Computational Linguistics, 2013, pp. 1–9.
- [11] A. X. Chang and C. D. Manning, “SUTime: A library for recognizing and normalizing time expressions,” in *Proceedings of the Eighth International Conference on Language Resources and Evaluation*. European Language Resources Association, 2012, pp. 3735–3740.
- [12] J. Strötgen and M. Gertz, “HeidelTime: High quality rule-based extraction and normalization of temporal expressions,” in *Proceedings of the 5th International Workshop on Semantic Evaluation*. Association for Computational Linguistics, 2010, pp. 321–324.
- [13] P. Mazur and R. Dale, “WikiWars: A new corpus for research on temporal expressions,” in *Proceedings of the 2010 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2010, pp. 913–922.
- [14] W. Xu, A. Ushio, and J. Camacho-Collados, “From discourse to dialogue: Multi-turn temporal resolution in conversational text,” in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2021.
- [15] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust speech recognition via large-scale weak supervision,” in *Proceedings of the 40th International Conference on Machine Learning*. PMLR, 2023, pp. 28 492–28 518.

- [16] B. Shi, X. Bai, and C. Yao, "An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 11, pp. 2298–2304, 2017.
- [17] M. Liao, Z. Wan, C. Yao, K. Chen, and X. Bai, "Real-time scene text detection with differentiable binarization," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34. AAAI Press, 2020, pp. 11 474–11 481.
- [18] J. Li, D. Li, S. Savarese, and S. Hoi, "BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models," in *Proceedings of the 40th International Conference on Machine Learning*. PMLR, 2023, pp. 19 730–19 742.
- [19] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, vol. 33. Curran Associates, 2020, pp. 1877–1901.
- [20] H. W. Chung, L. Hou, S. Longpre, B. Zoph, Y. Tay, W. Fedus *et al.*, "Scaling instruction-finetuned language models," in *Proceedings of the 41st International Conference on Machine Learning*. PMLR, 2022.
- [21] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei *et al.*, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.
- [22] B. T. Willard and R. Louf, "Efficient guided generation for large language models," in *arXiv preprint arXiv:2307.09702*, 2023.
- [23] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhunoye, Y. Yang *et al.*, "Self-refine: Iterative refinement with self-feedback," in *Advances in Neural Information Processing Systems*, vol. 36. Curran Associates, 2023.
- [24] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33. Curran Associates, 2020, pp. 9459–9474.
- [25] K. Shuster, S. Poff, M. Chen, D. Kiela, and J. Weston, "Retrieval augmentation reduces hallucination in conversation," in *Findings of the Association for Computational Linguistics: EMNLP 2021*. Association for Computational Linguistics, 2021, pp. 3784–3803.
- [26] B. Peng, M. Galley, P. He, H. Cheng, Y. Xie, Y. Hu, Q. Huang, L. Liden, Z. Yu, W. Chen, and J. Gao, "Check your facts and try again: Improving large language models with external knowledge and automated feedback," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, 2023.
- [27] C. Dwork and A. Roth, *The Algorithmic Foundations of Differential Privacy*, ser. Foundations and Trends in Theoretical Computer Science. Now Publishers, 2014, vol. 9, no. 3–4.
- [28] P. Marcelino, P. Lison, and L. Øvrelid, "Privacy-preserving neural text sanitization," in *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Student Research Workshop*. Association for Computational Linguistics, 2021, pp. 138–148.
- [29] D. Boneh and M. Franklin, "Identity-based encryption from the Weil pairing," in *Advances in Cryptology – CRYPTO 2001*. Springer, 2001, pp. 213–229.
- [30] S. Kulkarni, N. Bhagat, M. Fu, V. Kedigehalli, C. Kellogg, S. Mittal, J. M. Patel, K. Ramasamy, and S. Taneja, "Twitter heron: Stream processing at scale," in *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*. ACM, 2015, pp. 239–250.
- [31] R. T. Fielding and J. Reschke, "Hypertext transfer protocol (HTTP/1.1): Semantics and content," Internet Engineering Task Force, Tech. Rep. RFC 7231, 2014.